

M2 - Spring Core

MODULE 2

A deep dive into the heart of the Spring Framework — covering IoC, Dependency Injection, Bean Lifecycle, Scopes, Annotations, and Java-based Configuration.

Topics We'll Cover

01

Spring IoC Container

The core engine of Spring

03

Bean Lifecycle & Scope

How beans are created and managed

02

Inversion of Control & DI

IoC concepts and injection types

04

Annotations & Java Config

Stereotype annotations and @Configuration

Learning Objectives

After this session, students will be able to:

→ Explain the Spring IoC Container

Understand its role and responsibilities

→ Compare Injection Types

Constructor, Setter, and Field Injection

→ Differentiate IoC and Dependency Injection

Understand how they relate and differ

→ Configure Spring with Java

Use `@Configuration`, `@Bean`, and `@ComponentScan`

What is the Spring IoC Container?

The **IoC (Inversion of Control) Container** is the heart of the Spring Framework. It manages the full lifecycle of your application's objects.

Creates Beans

Instantiates Java objects automatically

Injects Dependencies

Wires objects together for you

Manages Lifecycle

Handles init and destroy callbacks

Reads Configuration

Processes annotations and config classes



IoC Container — How It Works

When the application starts, the **ApplicationContext** (Spring's IoC container) reads your configuration and creates all required beans, wiring them together automatically.



The container acts as the central authority — you define the classes, Spring handles the rest.

What is Inversion of Control?

Traditional Java

The **developer** controls object creation manually.

```
StudentRepository repo =  
    new StudentRepository();  
StudentService service =  
    new StudentService(repo);
```

Application controls all objects.

Spring IoC

The **framework** controls object creation. Developer only defines the classes — Spring creates and wires the objects.

```
@Service  
public class StudentService {  
    // Spring injects dependencies  
}
```

Spring owns the object creation process.

Traditional vs. Spring: Object Creation

1 Traditional Java

Everything is manually created in sequence:

- 🎬 StudentService creates StudentRepository
- 🎬 StudentRepository creates Database connection
- 🎬 Developer manages every dependency

2 Spring Framework

Spring Container owns the process:

- 🎬 Creates StudentRepository automatically
- 🎬 Creates StudentService and injects Repository
- 🎬 Application simply uses the ready service

📄 Spring owns the object creation process — developers focus on business logic, not wiring.



What is Dependency Injection?

A **dependency** is an object required by another object. For example, `StudentService` depends on `StudentRepository`. Instead of manually creating it, Spring **injects** it automatically.

Loose Coupling

Classes don't create their own dependencies

Easy Testing

Inject mock objects during unit tests

Easy Maintenance

Change implementations without touching consumers

RECOMMENDED

Constructor Injection

Constructor Injection is the **Spring-recommended approach**. Dependencies are passed through the constructor, making them mandatory and immutable.

```
@Service
public class StudentService {
    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }
}
```

✓ Immutable

Dependencies cannot change after creation

✓ Easy Testing

Pass mocks directly in constructor

✓ Mandatory

Forces all required deps at startup

Setter Injection

Setter Injection uses a public setter method to inject the dependency. Best suited for **optional dependencies**.

```
@Service
public class StudentService {
    private StudentRepository repository;

    public void setRepository(StudentRepository repository) {
        this.repository = repository;
    }
}
```

Advantages

- 🎬 Supports optional dependencies
- 🎬 Easy to modify after creation

Disadvantages

- 🎬 Object can remain partially initialized
- 🎬 Dependencies are mutable — less safe

NOT RECOMMENDED

Field Injection

Field Injection uses `@Autowired` directly on a field. While concise, it comes with significant drawbacks.

```
@Service
public class StudentService {
    @Autowired
    StudentRepository repository;
}
```

Advantages

- Simple, less boilerplate code

Disadvantages

- Difficult to unit test
- Hidden dependencies — not explicit
- Not recommended by Spring team

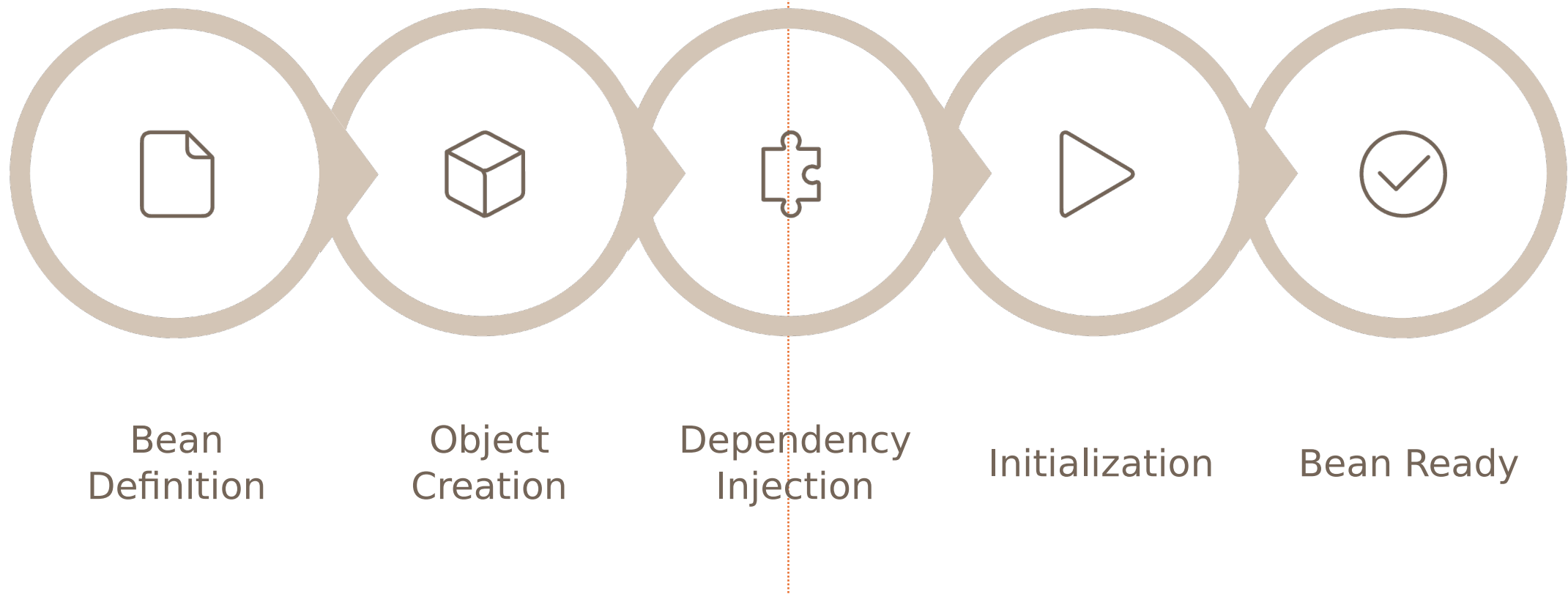
Dependency Injection — Comparison

Feature	Constructor	Setter	Field
Mandatory Dependency			
Unit Testing			
Spring Recommendation	Yes	Optional	Avoid
Immutable Object			

 **Best Practice:** Prefer Constructor Injection for all mandatory dependencies.

Spring Bean Lifecycle

Every Spring bean goes through a well-defined lifecycle — from definition to destruction. Understanding this helps you hook into key moments for initialization and cleanup.



Use `@PostConstruct` for initialization logic and `@PreDestroy` for cleanup when the application shuts down.

Bean Lifecycle — Key Phases Explained

1

Bean Creation

Spring instantiates the object from its definition.

2

Dependency Injection

All required dependencies are injected into the bean.

3

Initialization

`@PostConstruct` or `InitializingBean` runs custom setup logic.

4

Ready & Running

Bean is available throughout the application lifecycle.

5

Destroy

`@PreDestroy` runs cleanup before the bean is removed.

Bean Scope: Singleton vs. Prototype

Singleton (Default)

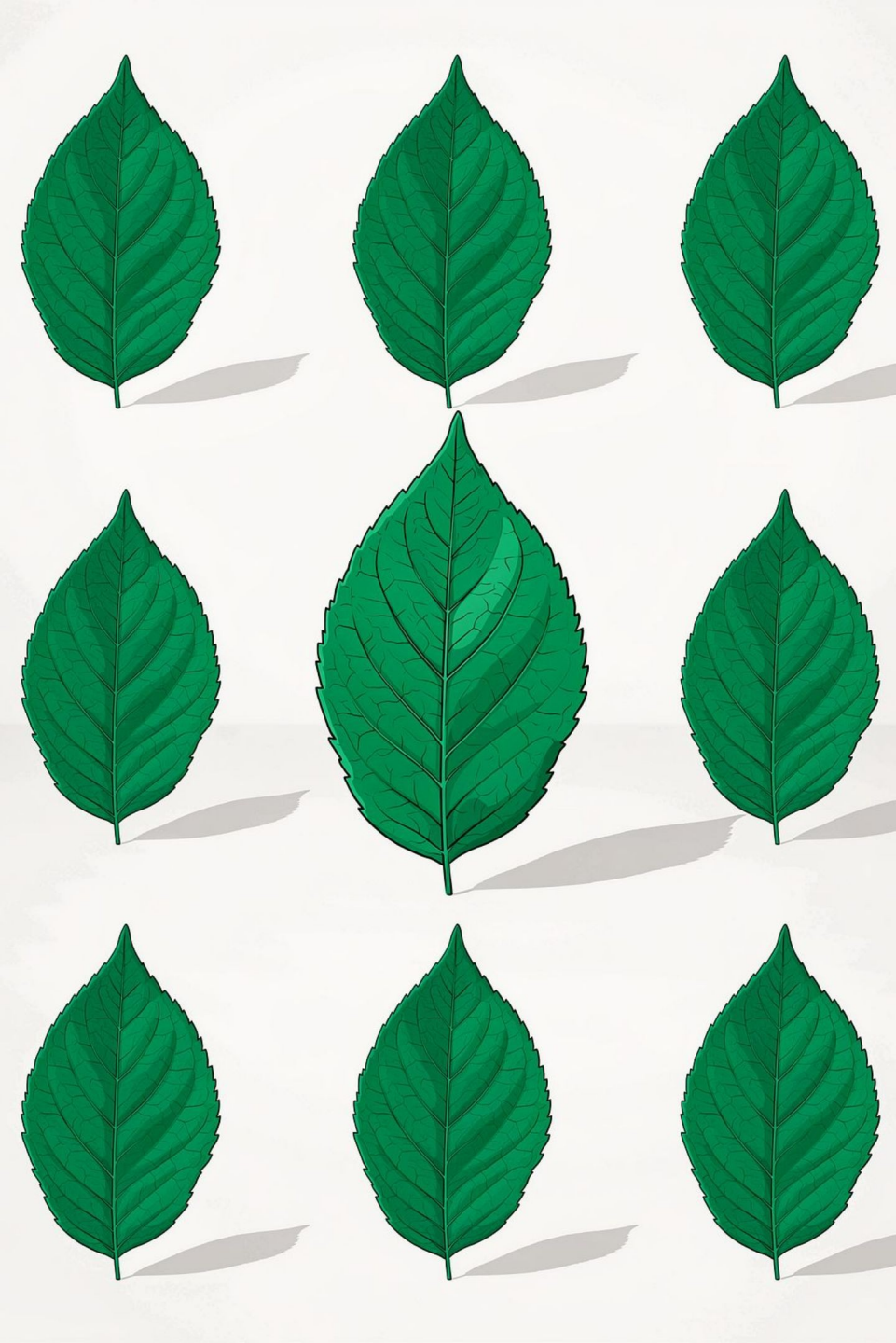
Only **one instance** is created for the entire application. All requests share the same object.

One Application → One StudentService Object

Prototype

A **new instance** is created for every request or injection point.

- Request 1 → Object A
- Request 2 → Object B
- Request 3 → Object C



Web Bean Scopes

Spring Web applications support additional scopes tied to HTTP context. These are **only available in Spring Web applications**.

Request Scope

One bean instance per HTTP Request. Created when the request starts, destroyed when it ends.

Request A → Bean A

Session Scope

One bean instance per User Session. Lives from user login until logout or session expiry.

User Login → Session Bean → Logout → Destroyed

Spring Stereotype Annotations



@Component

Generic Spring bean. Use for utility classes, helpers, calculators.



@Repository

Data access layer. E.g., `StudentRepository`, `EmployeeRepository`.



@Service

Business logic layer. E.g., `StudentService`, `PaymentService`.

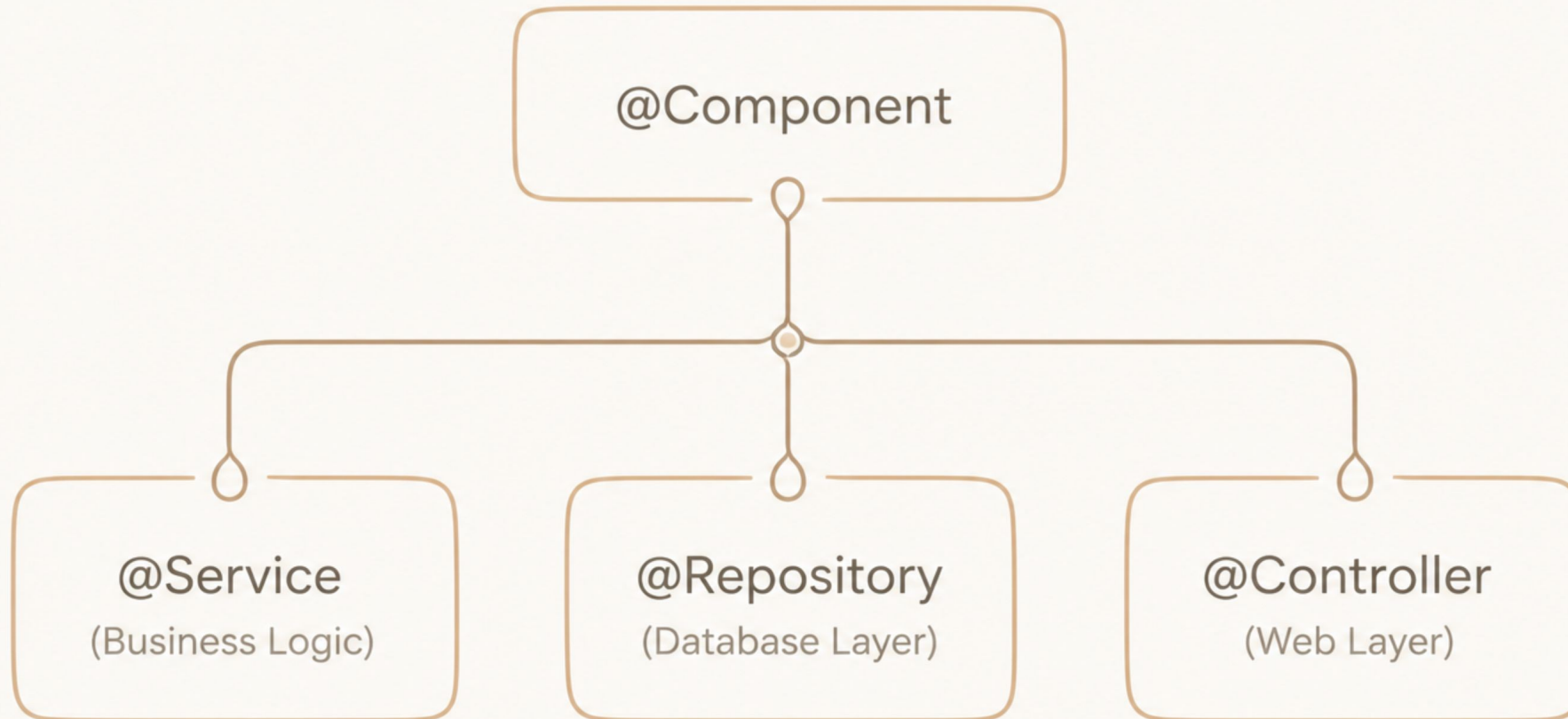


@Controller

Handles HTTP requests. E.g., `StudentController`, `LoginController`.

Component Annotation Hierarchy

All stereotype annotations are **specialized forms of @Component**. Spring treats them all as beans, but each carries semantic meaning for its layer.



Using the correct annotation improves code readability and enables layer-specific features like exception translation in `@Repository`.

Java-Based Configuration

Spring can be configured entirely with Java classes — no XML required. Three key annotations power this approach.

@Configuration

Marks a class as a Spring configuration source.

```
@Configuration
public class AppConfig { }
```

@Bean

Declares a Spring-managed bean inside a config class.

```
@Bean
public StudentService service() {
    return new StudentService();
}
```

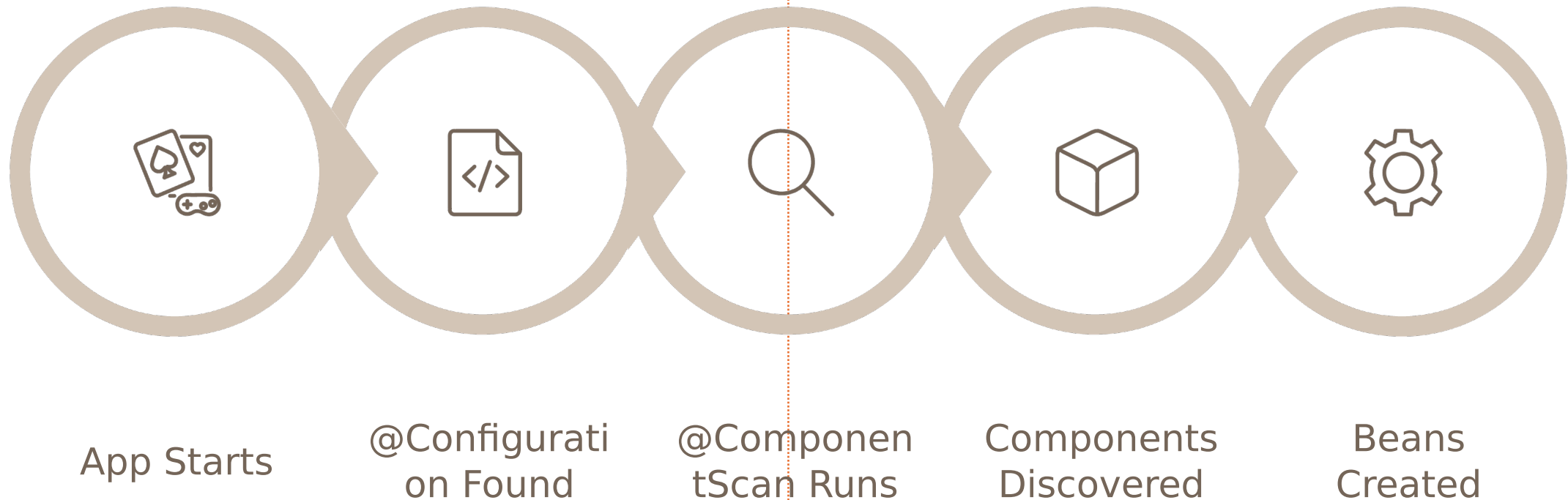
@ComponentScan

Automatically scans packages for `@Component` classes. No need to manually register each bean.

```
@ComponentScan("com.training")
```

Java Configuration — Startup Flow

When a Spring application starts with Java-based configuration, the container follows a predictable sequence to discover, create, and wire all beans.



This automated flow eliminates manual object creation and ensures all dependencies are resolved before the application serves any requests.

Best Practices

Prefer Constructor Injection

Ensures immutability and makes dependencies explicit and testable.

Use the Right Annotation

@Service for business logic, @Repository for data access, keep @Controller lightweight.

Avoid Field Injection

Hidden dependencies make testing difficult and violate best practices.

Default to Singleton Scope

Use other scopes only when your use case explicitly requires it.



Quick Reference: Injection Types

Constructor

Recommended. Mandatory, immutable, testable. Use for all required dependencies.

Setter

Optional. Use only for optional dependencies that may change after construction.

Field

Avoid. Simple but hides dependencies and makes unit testing very difficult.

Quick Reference: Bean Scopes

Singleton

Default. One shared instance per application context. Best for stateless services.

Prototype

New instance per injection. Use for stateful beans that must not be shared.

Request

One instance per HTTP request. Web apps only.

Session

One instance per user session. Web apps only. Destroyed on logout.

Module 2 — Summary

In this module, you learned the foundational concepts that power every Spring application:

01

Spring IoC Container

The engine that creates and manages all beans

03

Bean Lifecycle & Scopes

From creation to destruction; Singleton, Prototype, Request, Session

02

Inversion of Control & DI

Constructor, Setter, and Field Injection compared

04

Annotations & Java Config

@Component, @Service, @Repository, @Controller, @Configuration,
@Bean, @ComponentScan



Thank you